Next-Token Recommendation for Decentralized-Exchange Traders: A Project Plan

Gergely Madarász* Levente Tóth* Béla Jakab*

October 2026

Abstract

Which token will a trader pick up next? We treat trading accounts on Ethereum's Uniswap-style exchanges as users and tokens as items, and rank for each account the tokens it has never traded by how likely it is to trade them for the first time in the following two days. A feasibility study on 14 days of data (2.71 million swaps, 211,547 accounts, 20,014 tokens) fixed the task, the preprocessing, a leakage-controlled split on one global timeline, and a popularity baseline with recall@10 of 0.155 averaged over seven development points. The baseline shows what a model has to deal with: its score moves between 0.11 and 0.20 from one day to the next, it is the same list for every user, and 15 to 25% of the tokens users pick up did not exist when the list was made. No model results are reported here. The project proposes a two-stage method: candidate tokens from popularity and item-based nearest neighbours, re-ranked by a learned ranking model. It treats three questions as its core problems, each of which enters the ranker as a group of features: personalization from very short histories, the weighting of interactions by time, and cold start for newly launched tokens. It uses 12 weeks of earlier data that have not been examined, with 14 test points in place of one, and compares every model with the fixed baseline under a success criterion stated in advance.

Everything marked **done** has been run and can be reproduced from the project code; everything marked **planned** is the project work.

Where each grading criterion is answered. (a) Literature: Section 2 and the references. (b) Data: Section 3. (c) Preprocessing and split: Section 4. (d) Method fit: Section 6. (e) Baseline, metric and evaluation protocol: Section 5. (f) Description: the whole document, with a one-page summary in Section 1.

*Budapest University of Technology and Economics, Applied Machine Learning project work. Drafted with the assistance of an AI system (Claude Opus 5.5, Anthropic); see "Author contributions and provenance".

1 Summary

Task	Top-10 recommendation from implicit feedback. For each user, rank the tokens it has not traded; a hit is a token it trades for the first time in the next two days.
Data	Every swap on Uniswap V2 and V3 style pools on Ethereum, with the signing account and the pool's two tokens. Done: 14 days, 2.71 million swaps, 211,547 accounts, 20,014 tokens. Planned: the 12 weeks before the data examined so far.
Labels	The tokens actually traded later. Exact, free, no annotation.
Split	One global timeline. At each evaluation point all earlier data is the history and the next two days are the target. Development and test points lie in separate calendar periods.
Fixed baseline	The 10 most popular tokens (most distinct users in the history) that the user has not traded. No settings. Recall@10 = 0.155 on average over seven development points (range 0.114 to 0.201).
Main metric	Recall@10 averaged over users, reported per evaluation point and as a mean with its spread over points.
Proposed method	Two stages. Candidates: popular tokens and item-based nearest neighbours of the user's tokens. Ranking: a learned model (logistic regression, then gradient-boosted trees with a ranking loss) over popularity, neighbour, time and token features. No model results yet.
Core problems	(1) Personalization when most users have one token. (2) Time weighting: how much recent activity should count against older activity. (3) Cold start: tokens launched after the list was made.
Main risk	Day-to-day variation of the score is large. Two weeks are too short to judge any model against the baseline, so the project starts with more data.

Research questions.

1. When most users have traded a single token, what does item-based nearest neighbour personalization add to popularity?
2. Interest moves fast. How should interactions be weighted by their age, and how much history is worth keeping?
3. How much of the task is cold start (tokens that did not exist when the recommendation was made), and can refreshed lists and token features recover any of it?
4. Does a learned ranker over these signals beat their best fixed combination, on periods it was not trained on?
5. How many evaluation periods does it take to tell? (In the feasibility study a single evaluation point picked the wrong baseline.)

A note on the name. The idea started as "next-*pool*" recommendation. The data showed that routers, not traders, choose pools, so the item is now the token (Section 4.1). File names in the code still say "pool".

2 Literature

The plan rests on twenty-one references.

Depth of reading. Three papers were read in full: the source of the proposed method [3], the main reference for top-N evaluation [4] and the main reference for cold start [17]. They are reviewed in Section 2.4. The other eighteen were read at abstract level, five of them with their conclusions or main sections. The reference list marks the status of each entry.

2.1 Recommendation from implicit feedback

Trades are implicit feedback: there are positives (a trade) and no negatives (not trading a token is not dislike).

- Hu, Koren and Volinsky [1] treat such data as preferences with varying confidence and fit a weighted matrix factorization by alternating least squares (iALS).
- Rendle et al. [2] (BPR) optimize the ranking directly with a pairwise criterion.
- Sarwar et al. [3] introduced item-based nearest neighbours: recommend items that are similar, by their users, to the user's own items. This is the method proposed here (Section 2.4).
- Cremonesi, Koren and Turrin [4] show that a model built for rating prediction is not automatically good at top-N recommendation, and that popular items distort top-N evaluation (Section 2.4).
- Steck [5] (EASE) gives a linear item-item model with a closed-form solution that is competitive with deep models on sparse data.
- Hidasi et al. [6] (GRU4Rec) note that with short histories matrix factorization is inaccurate and practice falls back on item-to-item methods; they propose a recurrent network over the session.

2.2 Simple methods are hard to beat

- Dacrema, Cremonesi and Jannach [7]: of 18 neural recommenders from top venues, 7 could be reproduced, and 6 of those were beaten by simple neighbour or graph heuristics.
- Rendle et al. [8, 9]: a tuned dot product beats neural collaborative filtering, and tuned iALS is competitive with later methods.
- Anelli et al. [10] compare ten algorithms on three datasets: no consistent winner, no neural model best on any accuracy measure, and linear, neighbour and matrix factorization models consistently good.
- Ludewig and Jannach [11]: in session-based recommendation, nearest-neighbour methods "often perform equally well or significantly better" than recurrent networks.
- Li et al. [12] study next-basket recommendation, the setting closest to ours. Repeat items and explore items are different tasks and must be scored separately; against a frequency baseline, improvements of state-of-the-art methods are "modest or even absent".
- Ji et al. [13] show that the popularity baseline itself is usually evaluated carelessly: popularity should be measured at the time of the recommendation, and doing so on MovieLens improves it by 70% or more.

2.3 How to evaluate without fooling yourself

- Ji et al. [14]: leave-one-out splits per user leak the future, because a model trains on interactions that happened after the test interactions of other users. They call for one global timeline.
- Meng et al. [15]: the splitting strategy can change the ranking of models.
- Krichene and Rendle [16]: ranking the true item against a small sample of negatives is inconsistent with exact metrics and can reverse which model is better. Rank against all items.
- Schein et al. [17]: items without any interactions cannot be recommended by collaborative filtering and need item content (Section 2.4).

2.4 Three papers read in full

Sarwar, Karypis, Konstan and Riedl [3]: item-based collaborative filtering.

- **Problem.** User-based nearest neighbours search for similar users at request time. This scales badly, and with sparse data two users rarely share enough items for a reliable similarity.
- **Method.** Compute similarities between items from the columns of the user-item matrix, keep for each item only its k most similar items (the "model size"), and predict for a user from the items it has already rated, as a similarity-weighted sum. Three similarities are compared: cosine, Pearson correlation and adjusted cosine, which subtracts each user's mean rating.
- **Experiments.** MovieLens with 100,000 explicit ratings, 943 users with at least 20 ratings each, 1,682 movies. Random train/test splits, 80/20 in the main experiments. The metric is the mean absolute error (MAE) of predicted ratings.
- **Findings.** Adjusted cosine is clearly the best similarity. Item-based prediction is better than user-based at every neighbourhood size and sparsity level tried, but in the authors' words the improvement "is not significantly large" (MAE 0.726 against 0.732 with 80% training data). Accuracy improves up to about 30 neighbours and is flat after that. Keeping only 25 to 50 similar items per item, 1.9% to 3% of the catalogue, retains 96% to 98.3% of the accuracy of the full scheme and makes the system many times faster.
- **What we take from it.** The method itself, and the result that a short neighbour list per item is enough, which keeps the model small with tens of thousands of tokens.
- **What does not carry over.** (i) The paper predicts explicit ratings and scores them by MAE; we rank unseen items from binary data. Adjusted cosine has no meaning without ratings, so plain cosine is the natural choice here. (ii) Every user has at least 20 ratings; our median user has one token. (iii) The split is random, not by time. (iv) The paper precomputes similarities because "the item neighborhood is fairly static". In a token market that assumption is doubtful, which is one reason time weighting is a core problem (Section 6.3).

Cremonesi, Koren and Turrin [4]: top-N recommendation.

- **Problem.** Recommenders were mostly compared by rating error (RMSE), while users see a short list of the best items.
- **Method of evaluation.** Netflix (480,189 users, 17,770 items) and MovieLens (6,040 users, 3,883 items). Each held-out 5-star rating is ranked against 1,000 random items the user has not rated; recall@N and precision@N are reported. The test set is also split into the "short head" of the most popular items, which hold a third of all ratings (1.7% of the items on Netflix), and the "long tail".
- **Findings.** (i) There is no simple relation between RMSE and top-N accuracy. (ii) A non-personalized list of the most popular items is close to sophisticated models on the full test set: on MovieLens its recall@10 is about 0.29 against 0.28 for a strong factor model. (iii) On the long tail the popularity list collapses and the order of the personalized models changes. (iv) Two simple models that do not predict ratings at all do well. A truncated SVD of the matrix with missing entries as zeros (PureSVD) is the best model throughout. A cosine item-neighbour model that sums the similarities without normalizing them, with a shrinkage factor for pairs of items with few common users, is second on the full MovieLens test set.
- **What we take from it.** (i) Popularity as a serious reference in top-N evaluation. (ii) The form of our neighbour score: a plain sum of shrunk cosine similarities, which ranks an item

higher when many of the user's items point to it. (iii) Results are reported separately for the most popular tokens and for the rest (Section 5), so that a gain can be attributed.

- **What does not carry over.** The paper ranks each test item against 1,000 sampled items. Krichene and Rendle [16] later showed that sampled metrics can mislead, so we rank against all tokens. The split is random, not by time, and the data are explicit ratings. The neighbour model is the weaker of the two proposed models on the Netflix long tail, a warning that item neighbours are not automatically good away from the popular items.

Schein, Popescul, Ungar and Pennock [17]: cold start.

- **Problem.** Recommending items that no one has rated yet. Pure collaborative filtering "cannot help in a cold-start setting"; item content has to bridge from existing items to new ones.
- **Method.** A latent-class (aspect) model in which a person chooses a latent class and the class generates the *actors* of a movie instead of the movie itself. A new movie is "folded in" through its cast. The comparison is a naive Bayes classifier on the same content.
- **Experiments.** MovieLens with cast lists from the Internet Movie Database. The movies, not the ratings, are split at random: 331 of 1,682 movies have no observation in training. One of the three tasks is implicit rating prediction: will this person rate this movie at all.
- **Metrics.** Two ROC-style curves. The global curve (GROC) lets the system recommend more often to some users than to others. The customer curve (CROC) forces the same number of recommendations for every user.
- **Findings.** On implicit prediction the aspect model beats naive Bayes. A trivial heuristic that ranks users by how many movies they have seen is as good as the aspect model on the global curve (area 0.747 against 0.743). The two curves can disagree, and the authors ask that every cold-start evaluation include such heuristics.
- **What we take from it.** (i) Cold start needs attributes of the item, which for a token are few: its launch pool, its base currency, its deploying account and its first trades. (ii) Our metric gives every user a list of the same length, the setting of the customer curve, in which the user-activity heuristic has no advantage. (iii) The cold-start work will be compared with trivial heuristics, not only with a model.
- **What does not carry over.** The cold items are a random subset of an old catalogue with rich content. Ours are new in time, appear continuously and have almost no content. The paper does not address how early a new item can be recommended.

2.5 Learning to rank and two-stage recommendation

- Burges [20] describes the line from RankNet to LambdaRank to LambdaMART. RankNet learns from pairs of items of the same query with a logistic loss on the difference of their scores. LambdaRank scales the gradient of each pair by the change in NDCG that swapping the two items would cause, so that the model is trained for the ranking metric itself. LambdaMART is the same idea with gradient-boosted regression trees as the model.
- Covington, Adams and Sargin [21] describe the two-stage design used for YouTube: a candidate generator reduces millions of items to hundreds, and a separate ranking model with richer features orders them. Both are trained on implicit feedback. They report that giving the model the age of a training example as a feature removes a bias towards the past and lets it represent time-dependent popularity, and they stress that the training setup must not leak future information.

In our setting a "query" is a user at an evaluation point, and the items are the candidate tokens. The models of [21] are deep networks trained on billions of examples; we take the two-stage design from it, not the model class.

2.6 Recommendation for on-chain assets

- Choi et al. [18] build a graph recommender for NFTs from transactions plus image, text and price features, and report that item features help.
- Li et al. [19] (NFT.mine) apply a deep factorization machine to OpenSea data. They report AUC against classifiers, not a ranking metric on a time split, so the result is not comparable to ours.

Gap. Four web searches found recommenders for NFTs and general surveys of "blockchain-based recommender systems", which are about using a blockchain as infrastructure for privacy. **No paper was found that recommends fungible tokens from on-chain trading histories.** This is absence in a handful of searches, not a proof of novelty.

2.7 What the literature changes in the plan

Insight	Consequence for this project	Source
Only positives are observed	Binary interactions, ranking metrics, models built for implicit feedback	[1, 2]
Short histories favour item-to-item methods	Item nearest neighbours first; 56% of users have one item	[3, 6, 11]
Simple tuned models match complex ones	No neural model in the plan; tuned iALS and EASE as optional references	[7, 8, 9, 10]
Popularity is a strong and often mishandled baseline	Popularity is the fixed baseline, computed at the evaluation point from the history only	[12, 13]
Popularity depends on when it is measured	Time weighting is a core problem	[13]; own data
Popular items dominate top-N accuracy	Results also reported without the most popular tokens	[4]
A short neighbour list per item is enough	Top-k truncation of the similarities	[3]
Generate candidates cheaply, then rank them with a learned model trained for the ranking metric	Two-stage method with a learned ranker	[21, 20]
Repeat and explore are different tasks	"New tokens" is the primary task; repeats are scored separately	[12]
Per-user splits leak the future	One global timeline for all users	[14]
The split changes conclusions	Many evaluation points, spread reported	[15]; own data
Sampled metrics mislead	Every model ranks all tokens	[16]
Cold start needs content, and trivial heuristics as references	Token features and refreshed lists are a core problem	[17, 18]

Table 1: Insights from the literature and their consequence for the plan.

3 Data

3.1 The dataset (done)

Source	Ethereum mainnet, read through free public RPC endpoints (`eth.drpc.org` for logs, `ethereum-rpc.publicnode.com` for transactions). No key, no licence restriction.
Content	Every `Swap` event of Uniswap V2 and V3 style pools (including forks that keep the event, such as SushiSwap), the account that signed the transaction, and the two tokens of each pool, read from the pool contracts.
Window	14 days: blocks 26,049,600 to 26,150,399, 24 September to 8 October 2026.
Size	2,710,140 swaps in 1,965,611 transactions, by 211,547 signing accounts, on 24,630 pools holding 20,014 distinct tokens. About 440 MB on disk as Parquet.
Access	Regenerated by the download script `dex_swaps.py` in about two hours.

Table 2: The dataset of the feasibility study.

One row per swap: `block`, `tx_index`, `log_index`, `timestamp`, `tx_hash`, `tx_from` (the signer, our user), `tx_to`, `pool`, `version`, `sender`, `recipient`, `amount0`, `amount1`. The sign of the amounts tells which token the trader paid and which it received.

Validation. 99.1% of blocks contain a swap; there are no duplicate events (`tx_hash`, `log_index`) and no missing values; the tokens of 24,620 of the 24,630 pools could be read, covering 99.98% of swaps; daily volume is even, with 164,000 to 225,000 swaps and 21,700 to 30,500 users on each full day.

Not in the data. Other exchange designs (Uniswap V4, Curve, Balancer), prices, and anything about a user beyond its trades. A user is an address; one person can have several.

3.2 Descriptive statistics (done)

Property (raw data, pools as items)	Value
Distinct user-pool pairs	719,752 (0.014% of the matrix)
Median pools per user	1
Users with one pool	56%
Users with at least 5 pools	21,990
Pools with fewer than 5 users	70%
Share of swaps in the 10 / 100 most used pools	29% / 59%
Share of swaps by the 100 / 1,000 most active users	27% / 57%
Transactions touching more than one pool	20%, containing 41% of the swaps

Table 3: Sparsity and skew of the raw interaction data.

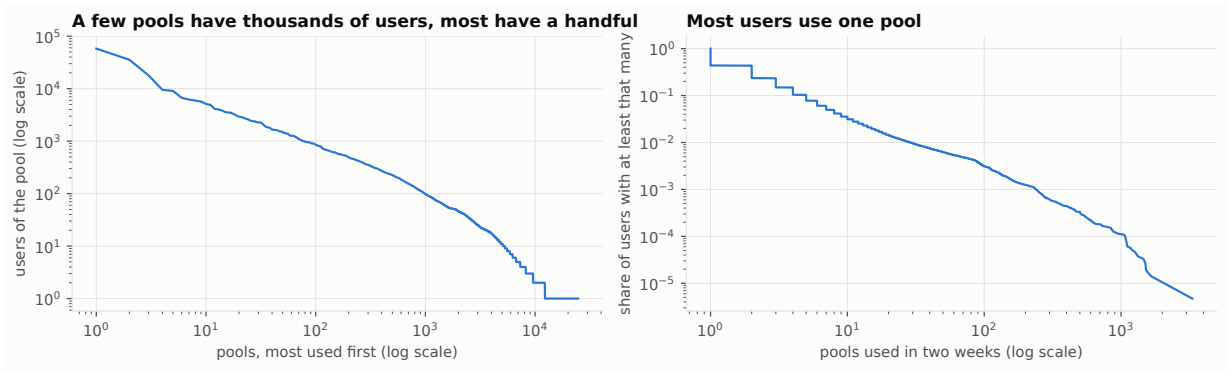


Figure 1: Left: users per pool, most used pool first. Right: share of users with at least a given number of pools. Both axes are logarithmic.

Everything is heavily skewed (Figure 1). There are plenty of events, but the number of users with enough history to personalize for is in the thousands, and popularity is bound to be a strong reference.

3.3 Features and labels

After the preprocessing of Section 4.1 the working data is a table of *trade legs*: one row per (transaction, token) with `block`, `user`, `token` and `side` (buy or sell).

Users with at least one token	157,060
Tokens	19,631
Median tokens per user	1
Users with at least 2 / 5 tokens	47,280 / 11,000
Tokens with at least 5 / 100 users	4,655 / 606
Share of buys among the legs	52%

Table 4: The user-token data after preprocessing.

- **Features (core).** The binary user-token matrix of the history: who traded what before the evaluation point. Derived from it: token popularity overall and in the last day, and token-token co-usage.
- **Features (planned extensions).** Token age and first-seen time, user history size, buy or sell side, time since the user's last trade, and for cold start the token's launch pool and deployer.
- **Label.** A pair (user u, token i) is positive at evaluation point t if u buys or sells i in the two days after t and never traded it before t. Labels come from the chain itself: exact, free, and available for any period.
- **Size of the supervised problem.** At each evaluation point 6,700 to 8,600 users are evaluated, with 23,000 to 27,000 positive pairs (Table 7).

3.4 The data of the project (work package 1)

Two weeks are enough to see the pattern and not enough to rely on it (Section 5.4). The project uses **12 weeks of earlier data**, downloaded with the existing, resumable script.

Part	Blocks	Approximate dates (2026)	Status and use
Project data, 12 weeks	25,344,000 to 25,948,799	18 June to 10 September	To download. All tuning, selection and testing
Two weeks before the feasibility window	25,948,800 to 26,049,599	10 to 24 September	Downloaded and looked at in a check of the baseline on a longer window. Not used for evaluation
Feasibility window	26,049,600 to 26,150,399	24 September to 8 October	Downloaded. Exploration only

Table 5: The data of the project and the data examined so far.

- **Expected size:** about 16 million swaps and 2.5 to 3 GB, extrapolated from the current 1.35 million swaps per week.
- **Cost:** 20 minutes to 1.5 hours of download per week, measured on the weeks downloaded so far; no fee.
- **Why earlier data:** all of it exists today, so nothing in the project depends on waiting for new blocks. And none of it has been examined, which makes a clean test set possible (Section 4.2).

4 Preprocessing and split

4.1 Preprocessing (done)

The central decision: the item is the token, not the pool.

- A fifth of all transactions pass through more than one pool. A user swapping ETH for a token may be routed through ETH/USDC and then USDC/TOKEN. The user chose the token; the router chose the pools.
- Of the user-pool pairs that count as "new" at pool level, only 73% are a token new to the user. 15.5% are pools of two base currencies and 11.7% are another pool for a token the user had already traded.
- Each transaction is therefore reduced to the tokens its sender ended up selling and buying; tokens that were only passed through are dropped (`pool_recsys.trade_legs`, tested). 97.7% of transactions keep at least one token.

Issue found in the data	Decision
Base currencies are in almost every trade	Tokens traded by more than 10% of users are removed. This separates three tokens (wrapped ETH 81%, USDC 40%, USDT 29%) from the fourth most common (5%). Decided on the train period only.
Arbitrage cycles end where they start	They leave no bought or sold token and drop out, which removes much pure bot activity without a bot filter.
A few accounts make a large share of the swaps	Interactions are binary, and every metric is an average over users, so an account with 10,000 swaps counts as much as one with 2.
Half of what users do next is a repeat	New tokens and all tokens are separate tasks. Only the first is pursued.
Tokens that did not exist in the history	Counted as misses, not removed, so that scores reflect the real task.
Buying and selling are different acts	Both count in the history. A variant with first-time buys only as the target is reported.

Table 6: Preprocessing decisions.

4.2 The split

Principle. A recommender has no long training phase to protect. The unit of the split is the *evaluation point*: a moment t on one global timeline, the same for all users. Everything before t is the training history; the two days after t are the target. Points are assigned to train, development or test by calendar period.

In the feasibility study (done). Train is days 1 to 10, dev days 11 and 12, test days 13 and 14. Seven rolling development points, after day 4 to day 10, were used instead of one; none of their targets reaches into the test days. The single test point is after day 12 (Table 7).

Evaluation point	Use	Users in history	Tokens in history	Users evaluated	New pairs to predict	Token absent from history
after day 4	dev	60,994	9,933	8,037	27,349	14.9%
after day 5	dev	72,173	10,699	7,351	26,438	21.9%
after day 6	dev	84,534	11,723	7,073	26,223	21.6%
after day 7	dev	93,907	12,572	6,684	25,972	20.3%
after day 8	dev	103,268	13,240	7,237	23,573	20.1%
after day 9	dev	110,649	14,701	8,131	25,650	24.8%
after day 10	dev	119,785	17,530	8,403	24,583	24.0%
after day 12	test	138,501	18,701	8,604	23,138	21.7%

Table 7: Sizes at each evaluation point of the feasibility study.

Two weaknesses of this split, both to be removed. Neighbouring targets overlap by one day, so the seven points are not independent. And the test point is no longer untouched: it was looked at during exploration (Section 9).

For the project (planned). On the 12 weeks of project data, with non-overlapping two-day targets:

Part	Period (2026)	Points	Use
Burn-in	18 June to 2 July	none	History only, so that every point has at least 14 days behind it
Dev A (tune)	2 to 22 July	10	Fit neighbour settings, time weights and blend weights; train the ranker
Dev B (select)	22 July to 13 August	11	Choose between models
Test	13 August to 10 September	14	Scored once, after the models and their settings are frozen and committed

Table 8: The split of the project.

- **The order is chronological:** tuning, then selection, then test. No setting is chosen on data that lies after the test period.
- **The test period has not been examined.** It ends where the data looked at so far begins.
- **One caveat.** The plan itself, including the choice of the three core problems, was formed on the later exploration data. The test period is therefore untouched, but it is not later in time than everything the authors have seen.
- A burn-in of 14 days also removes a confound of the feasibility study, where early points had 4 days of history and late ones 10.

4.3 What the split shows about the task (done)

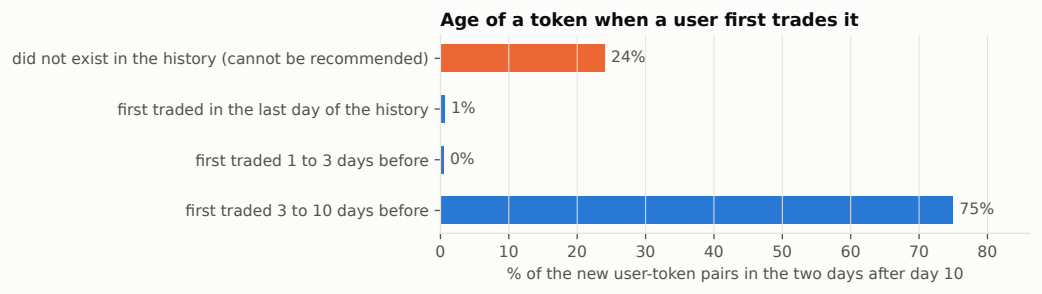


Figure 2: New user-token pairs in the two days after day 10 (24,583 pairs), by the age of the token at the evaluation point.

- **15 to 25% of the tokens users pick up next did not exist in the history.** They were launched inside the two target days. No model built on past interactions can recommend them, which caps recall well below 1.
- **Almost all the rest are established tokens** (Figure 2). 75% of the new pairs go to tokens that had traded for at least three days, about 1% to tokens that appeared in the last three days of the history. A token is picked up at launch or once it is established, with little in between.
- **Adoptions are spread out.** The 10 most picked tokens cover 14% of the new pairs, the 100 most picked 39%.

4.4 Leakage controls

Risk	Control
A random or per-user leave-one-out split shows a model other users' future [14]	One global point in time per evaluation, for all users.
Users or items of the future enter the model	Users and items are indexed from the history only (`pool_recsys.build_task`, tested). Future-only tokens count as misses.
Item statistics computed on all data	Popularity, recent popularity and (planned) token age are computed from the history only.
The base-currency list uses the future	Decided on the first ten days. The same three tokens result from any cut.
Recommending what the user already has	The user's history is removed from the ranking.
Sampled negatives distort the comparison [16]	Every model ranks all tokens of the history.
Tuning on test	Settings are chosen on Dev A, models on Dev B. The test period is scored once from a frozen commit.
The ranker learns from labels that lie after the points it is scored on	It is trained on Dev A points only, all of which end before Dev B and the test period begin. The features of every training example use the history before that example's point.
Random cross-validation mixes rows of one period into training and validation	Hyperparameters of the ranker are chosen by holding out whole evaluation points.
Overlapping targets make points look independent	Non-overlapping two-day targets in the project.
Pool tokens were read today, not at swap time	A pool's two tokens are immutable. Not a leak.
Evaluated users are chosen by their future (they trade something new)	This defines who is scored, not what a model sees. The metric reads: given that a user picks up something new, was it in the list.
One person, several accounts	Not handled. It makes users look less active than they are and cannot leak labels.
Trades before the data starts	A token traded before the first block of the data looks "new". The longer history reduces this; it is not removed.

Table 9: Leakage risks and controls.

5 Baseline, metric and evaluation protocol

5.1 Task

- **Primary:** for each user, rank the tokens it has not traded. A hit is a token it trades for the first time in the next two days.
- **Secondary, reported only:** rank all tokens, repeats included.

5.2 Metrics

Metric	Definition	Why
Recall@10 (primary)	Share of the user's new tokens that are in its list of 10, averaged over users	A user can act on a short list. Averaging over users keeps a few very active accounts from dominating.
NDCG@10	As recall, with more credit for hits near the top	Position matters in a list.
Hit rate@10	Share of users with at least one hit	The plain reading: for how many users was the list any use.
Coverage	Share of all tokens that appear in anyone's list	Detects a model that shows everyone the same few tokens.
Spread over evaluation points	Mean, lowest and highest value	The period is the main source of uncertainty.

Table 10: Metrics. Tokens absent from the history stay in the denominator of recall and NDCG.

5.3 Protocol

1. At each evaluation point, fit every model on the history only.
2. For every evaluated user (at least one token in the history, at least one new token in the target), score *all* tokens of the history, remove the user's own, and take the top 10.
3. Score the lists against the next two days.
4. Report each metric per point, and its mean, lowest and highest value.
5. **Comparison with the baseline is paired by evaluation point:** the mean difference in recall@10, a bootstrap interval over points, and the number of points won. With 14 test points, 12 wins are needed for a two-sided sign test at the 5% level.
6. Within one point, a bootstrap over users gives the sampling interval; the function is `bootstrap_mean` in `pool_recsys.py`.
7. Results are also reported by history size: 1, 2 to 4, and 5 or more tokens.
8. Following [4], recall@10 is also reported with the most popular tokens removed from the target (the tokens that hold a third of the interactions of the history), so that a model cannot look good on popular tokens alone.

Success criterion, fixed in advance. The proposed method improves mean recall@10 over the fixed baseline on the test points, with a bootstrap interval of the paired difference that excludes zero.

5.4 The fixed baseline and how it performs (done)

Popularity: recommend the tokens with the most distinct users in the history, skipping those the user has traded. It has no hyperparameter and is not personalized. Following [13], it is computed at the evaluation point, from the history only.

	Development (7 points)					
New tokens	Recall@10	Lowest	Highest	NDCG@10	Hit rate	Coverage
Popularity (fixed baseline)	**0.155**	0.114	0.201	0.094	0.271	0.7%
Recent popularity (last day only)	0.156	0.100	0.197	0.096	0.269	0.6%
	Test point (after day 12)					
	Recall@10			NDCG@10	Hit rate	
Popularity (fixed baseline)	**0.202**			0.111	0.289	
Recent popularity (last day only)	0.173			0.098	0.254	

Table 11: Performance of the fixed baseline and of its recency variant.

Recall@10 after day	4	5	6	7	8	9	10	12 (test)
Popularity	0.201	0.185	0.161	0.157	0.128	0.114	0.142	0.202
Recent popularity	0.197	0.186	0.150	0.155	0.122	0.100	0.181	0.173

Table 12: The baseline at each evaluation point.

- **It finds a real share:** 16% of first-time tokens on average, and at least one right token for about one user in four.
- **It depends strongly on the period:** 0.11 to 0.20. The sampling interval over users at one point is about ±0.007, so the period effect is several times larger.
- **Recency helps on some days and not on others.** At the single point first examined (after day 10) recent popularity wins by 0.04 with tight intervals. Over seven points it wins twice, and at the test point it loses by 0.03. One evaluation point would have chosen the wrong baseline, and how to weight time is an open question (Section 6.3).
- **It is not a recommender:** under 1% of tokens ever appear in a list, and every user gets the same one.
- **It is weakest where there is most to personalize from.** Pooled over the development points, recall@10 is 0.179 for users with one token in their history (16,077 evaluations), 0.183 with two to four (15,438) and 0.117 with five or more (21,401).
- **The level does not depend on details of the task.** With only first-time buys as the target the baseline scores 0.153, and without the 5% of users with the largest histories 0.161.
- **With repeats included,** a user's own history (0.270) beats popularity (0.166), as [12] predicts. That task is not pursued.

In the project the baseline definition stays as it is, with the history taken as the 14 days before the evaluation point so that every point is comparable. It will be re-measured on all development points before any model is compared with it.

6 Method and why it fits

6.1 Proposed method

Two stages [21]. The first produces candidate tokens and scores for each user from popularity and item-based nearest neighbours [3]. The second is a learned model that ranks the candidates

(Section 6.4). The first stage with a fixed weight is a complete recommender on its own and is the reference the ranker has to beat.

Let U_i be the set of users who traded token i in the history and H_u the set of tokens user u traded.

- **Similarity of two tokens:** the cosine of their user sets with a shrinkage term λ that distrusts tokens with few users,

$$s_{ij} = \frac{|U_i \cap U_j|}{\sqrt{|U_i|\,|U_j|} + \lambda}.$$

  For each token only the k most similar tokens are kept, which [3] found to cost little accuracy.

- **Neighbour score of a candidate token:** $r_{ui} = \sum_{j \in H_u} s_{ji}$, the similarity of i to everything the user has traded. The sum is not normalized, following [4]: a token that many of the user's tokens point to ranks higher.

- **Fixed combination:** $w\,\tilde{p}_i + (1-w)\,\tilde{r}_{ui}$, where $p_i = |U_i|$ is the popularity of the baseline and the tilde scales a score to a maximum of 1 per user. With $w = 1$ this is the fixed baseline, so the baseline is a special case of the method.

Step	Model	Settings to tune on Dev A	Status
0	Popularity (fixed baseline)	none	Done
1	Item nearest neighbours	k, shrinkage λ	Planned
2	Combination with popularity	weight w	Planned
3	Time weighting of interactions (core problem 2)	half-life of the decay, length of the history	Planned
4	Cold start with refreshed lists and token features (core problem 3)	refresh interval, feature set	Planned
5	Learned ranker over the scores and features of steps 0 to 4	regularization; number, depth and learning rate of the trees	Planned

Table 13: The method, in steps. Each step is judged against the previous one.

Status. Only step 0 has results in this plan. Steps 1 and 2 are implemented and tested in `pool_recsys.py` and were run with untuned settings during exploration. Those runs are not reported: seven overlapping evaluation points are too few to judge a model against a baseline that itself moves between 0.11 and 0.20 (Section 9).

6.2 Why this fits the task and the data

Property of the task	What it calls for	How the method answers
Only positives, no ratings	Implicit-feedback models and ranking metrics [1, 2]	Binary matrix; co-usage similarity; recall and NDCG
The median user has one token	Models that work from a single item [6, 11]	Item-to-item neighbours need one item; user factors would be fitted on almost nothing
Extreme skew; popularity is strong	Keep popularity in the model [12, 13]	Popularity is one of the two terms of the score
Thousands of tokens with a handful of users	Distrust thin evidence	Shrinkage in the similarity; top-k truncation
13,000 to 19,000 items, about 100,000 users	Nothing heavy is needed [7, 10]	Sparse linear algebra on one machine
Fast-moving market	Refit at every evaluation point; weight by time	The first stage has no training phase; time weights enter the same formulas
Several signals whose best mix differs between users	Learn the combination instead of fixing one weight [20]	The ranker sees all signals together with the size of the user's history
Labels are exact and free at every evaluation point	Supervised learning is possible without annotation	Each development point is a labelled training set for the ranker

Table 14: Fit between the task and the method.

6.3 The three core problems

1. Personalization from very short histories.

- **What the data shows.** After preprocessing the median user has one token; 47,280 users have at least two and 11,000 at least five. The baseline is weakest for the users with the longest histories (0.117 against about 0.18, Section 5.4), which is where a personalized model has the most to work with.
- **Open question.** Whether co-usage carries enough signal for a user with a single token, and how much weight popularity should keep.
- **Work.** Tune k, λ and w on Dev A. Report every result by history size (1, 2 to 4, 5 or more tokens), since an average over all users can hide that the method helps one group and not another.

2. Time weighting.

- **What the data shows.** The baseline moves between 0.11 and 0.20 from day to day. Popularity of the last day wins by 0.04 at one point and loses at five of seven. A token is picked up at launch or once it is established (Figure 2). In the feasibility study the history also grows from 4 to 10 days across the points, so history length and calendar time are mixed.
- **Open question.** How fast the value of an interaction decays, and whether a longer history helps or adds stale information. With 12 weeks of data this decides whether more history is an advantage at all.
- **Work.** Give an interaction of age a the weight $2^{-a/h}$ with half-life h, in three places: the popularity p_i, the similarity s_{ij} and the user's profile H_u. Tune h and the length of the history on Dev A. All-time popularity ($h \to \infty$) and popularity of the last day are the two ends of this range, and both are reported as references [13].

3. **Cold start.**

- **What the data shows.** 15 to 25% of the tokens users pick up next did not exist in the history (Table 7). No model that knows only past interactions can recommend them. Under the protocol of Section 5 token features cannot reach them either, since the token does not exist when the list is made.
- **Open question.** How much of this part becomes reachable if lists are refreshed, and how early a new token can be ranked sensibly.
- **Work.** A second protocol in which lists are rebuilt every few hours inside the target window, with the baseline refreshed in the same way so that the comparison stays fair. Token features that exist from the first minutes [17, 18]: age, the launch pool and its base currency, the deploying account, and the number of users in the first hour. As [17] ask, the comparison includes trivial heuristics, such as showing every user the new tokens with the most early users.
- **Measure.** The share of the cold-start pairs that enter a list, next to recall@10 under the refreshed protocol.

6.4 The learned ranker

Candidates. For each user, the union of the top tokens by popularity and by neighbour score, a few hundred in total. The share of the true new tokens that are among the candidates (candidate recall) is measured first, since it is the ceiling of anything the ranker can reach.

Training examples. One row per (evaluation point, user, candidate token), with the label of Section 3: 1 if the user trades the token for the first time in the two days after the point. Each Dev A point contributes about 8,000 users and their candidates. Positives are a small minority of the rows.

Feature group	Features of a (user, candidate token) pair	Core problem
Popularity	Users of the token in the whole history, in the last day, and with several half-lives	2
Neighbours	Neighbour score r_{ui}, the largest single similarity, the number of the user's tokens that point to the candidate; plain and time-weighted	1, 2
User	Number of tokens in the history, time since the last trade, share of buys	1
Token	Age, number of users in its first hour and first day, base currency of its launch pool	3

Table 15: Features of the ranker. All are computed from the history before the evaluation point.

Models, in order of complexity.

1. **Logistic regression.** A learned version of the fixed combination, with a handful of parameters. It shows what learning the weights is worth before any non-linearity is added.
2. **Gradient-boosted trees with a logistic loss** on single rows. Trees can represent that the best mix of signals depends on the size of the user's history.
3. **Gradient-boosted trees with a LambdaMART objective** [20], where the rows of one user at one point form a group and the gradients are weighted by the change in NDCG@10.

Training and validation.

- **Fit** on the ten Dev A points.
- **Hyperparameters and early stopping** by cross-validation that holds out whole evaluation points. The ten points are ten market situations, not millions of independent rows, and a random split of rows would hide overfitting to a period.
- **Selection** between the fixed combination and the three rankers on Dev B, by the paired comparison of Section 5.
- **Test** once, with everything frozen.

What the ranker is for.

- **One model for the three core problems.** The weight between popularity and neighbours, the half-life and the use of token features are learned together instead of tuned one after the other.
- **Ablation.** Training the ranker without one feature group at a time measures what neighbours, time and token features each contribute. This is the main analysis of the project.
- **A fair reference.** The ranker is compared with the best fixed combination of steps 2 and 3, not only with the baseline, so that the gain from learning is separated from the gain from the signals.

6.5 What is left out, and why

- **Matrix factorization.** User factors would be fitted on one or two interactions for most users. Tuned iALS [1, 9] and EASE [5] are optional reference models, run if time allows.
- **Sequence and neural models.** Most users have one or two tokens, so there is little sequence, and [7, 10, 11] give no reason to expect a gain at this size and sparsity.
- **Honest ceiling.** The baseline misses about five of every six tokens a user will pick up, and up to a quarter of them are out of reach without the cold-start work. High absolute scores are not to be expected from any method.

7 Work plan and outlook

#	Work package	Output	Depends on
1	**Data.** Download the 12 earlier weeks; repeat the validation on each week	12 weeks of validated swaps and trade legs	none
2	**Protocol.** Rolling evaluation with non-overlapping targets, a 14-day burn-in, paired comparison over points; re-measure the baseline	Baseline table on 21 development points	1
3	**Core problem 1: nearest neighbours.** Tune k, λ and the weight w on Dev A, confirm on Dev B; results by history size	First model comparison with the baseline	2
4	**Core problem 2: time weighting.** Half-life and history length for popularity, similarity and user profile	Time-weighted model; effect of history length	3
5	**Core problem 3: cold start.** Refreshed-list protocol; token features	Share of the cold-start part that becomes reachable	2
6	**Learned ranker.** Candidate recall; logistic regression, then boosted trees with a logistic and a LambdaMART loss; ablation by feature group	Ranker and its comparison with the best fixed combination on Dev B	3, 4, 5
7	**Test and write-up.** Freeze the models, score them once on the test points; error analysis; report	Final report	3 to 6

Table 16: Work packages. Optional, if time allows: the reference models iALS and EASE, and a separation of automated accounts by behavioural features.

Work	What it can and cannot deliver	Basis
More weeks, non-overlapping points	No gain in score; the precondition for judging any model	Day-to-day range of 0.11 to 0.20
Nearest neighbours with popularity	Personalized lists and far higher coverage; whether recall improves is the first thing to establish	Baseline coverage under 1%; [3, 11, 12]
Time weighting	A more stable model across periods; needed before 12 weeks of history are of use	Recent popularity wins on some days only; [13]
Cold start	The largest part no interaction model can reach, and the hardest: a model has minutes to hours of information	15 to 25% of the target; [17, 18]
Learned ranker	A gain over the best fixed combination where the best mix of signals differs between users; bounded by candidate recall	Baseline recall differs by history size (0.18 against 0.12); [20, 21]

Table 17: What to expect from each part of the work.

Outlook beyond the project.

- **More of the market:** Uniswap V4, Curve, Balancer and aggregators, so that a token bought elsewhere no longer looks new.
- **Other chains:** Base or Solana, where token launches are far more frequent and the cold-start share should be higher.
- **Linking addresses** of one person, which would lengthen the short histories that limit personalization.
- **Horizon and list length:** one day and one week instead of two days; recall@20 and @50.

- **The evaluation question on its own:** how many evaluation periods a fast-moving domain needs before a model ranking is stable.

8 Risks

Risk	Likelihood	Mitigation
The method does not beat the baseline	Medium	Popularity is a strong reference in this literature [12]. A null result on 14 clean test points is still a reportable finding, and the method contains the baseline as a special case.
Period effects stay larger than model effects	High	Paired comparison per evaluation point; many points; spread always reported.
Public endpoints become slow or unavailable	Medium	The download is resumable and chunked, and a rate limit only slows it down. All the data already exists.
Bots dominate the evaluated users	Medium	User-averaged metrics, binary interactions, results by history size.
The ranker overfits a few development periods	High	Ten training points are ten market situations. Logistic regression first; few features; shallow trees; validation on held-out points; selection on Dev B only.
The cold-start work does not fit in the time	Medium	It depends only on the protocol, not on the other models, and can be reported as a separate result.
Usefulness and harm	n/a	Recommending speculative tokens is not obviously desirable. The project is framed and reported as a study of trading behaviour and of evaluation, not as a product. Only public, pseudonymous data is used.

Table 18: Risks and mitigations.

9 Limitations of the evidence so far

- **Two weeks, one market phase.** Seven development points with overlapping targets, one test point.
- **Only the baseline has reported results.** Untuned versions of the neighbour model were run during exploration and are left out of this plan, because seven points cannot separate a model effect from a period effect.
- **The test point is not untouched.** During exploration the baselines and the untuned neighbour model were scored at the points after day 11 and day 12. This is why the project uses a test period that has not been examined.
- **Earlier points have shorter histories** (4 days against 10), so time and history length are mixed in the day-to-day variation.
- **The 10% threshold for base currencies** was set by inspection and its sensitivity was not tested, though the gap between the third token (29%) and the fourth (5%) is wide.
- **Only Uniswap V2 and V3 style pools.**
- **Most of the literature was read at abstract level;** three papers were read in full (Section 2).

10 Reproducibility

- The project code consists of the download script (`dex_swaps.py`), the preprocessing, models and metrics (`pool_recsys.py`), their tests, and the notebook that produced every num-

ber and chart in this plan (`05_pool_recommendation_data_and_baseline.ipynb`). The notebook also holds the exploratory model runs that this plan does not report.

- `python dex_swaps.py --start 26049600 --blocks 100800` downloads the current window in about two hours; the notebook then runs in about two minutes.
- `pool_recsys.py` is covered by 18 tests.
- Random seeds are fixed.

Author contributions and provenance

G. M., L. T. and B. J. chose the topic together, developed the code and reviewed the process. An AI system (Claude Opus 5.5, Anthropic) assisted with the code, ran the experiments and drafted this text under the authors' direction. All three authors are responsible for the content.

References

[1] Y. Hu, Y. Koren, C. Volinsky. Collaborative filtering for implicit feedback datasets. *ICDM*, 2008. http://yifanhu.net/PUB/cf.pdf. [Read: abstract.]

[2] S. Rendle, C. Freudenthaler, Z. Gantner, L. Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. *UAI*, 2009. arXiv:1205.2618. [Read: abstract.]

[3] B. Sarwar, G. Karypis, J. Konstan, J. Riedl. Item-based collaborative filtering recommendation algorithms. *WWW*, 2001. [Read in full.]

[4] P. Cremonesi, Y. Koren, R. Turrin. Performance of recommender algorithms on top-N recommendation tasks. *RecSys*, 2010. [Read in full.]

[5] H. Steck. Embarrassingly shallow autoencoders for sparse data. *WWW*, 2019. arXiv:1905.03375. [Read: abstract.]

[6] B. Hidasi, A. Karatzoglou, L. Baltrunas, D. Tikk. Session-based recommendations with recurrent neural networks. *ICLR*, 2016. arXiv:1511.06939. [Read: abstract.]

[7] M. F. Dacrema, P. Cremonesi, D. Jannach. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. *RecSys*, 2019. arXiv:1907.06902. [Read: abstract.]

[8] S. Rendle, W. Krichene, L. Zhang, J. Anderson. Neural collaborative filtering vs. matrix factorization revisited. *RecSys*, 2020. arXiv:2005.09683. [Read: abstract.]

[9] S. Rendle, W. Krichene, L. Zhang, Y. Koren. Revisiting the performance of iALS on item recommendation benchmarks. arXiv:2110.14037, 2021. [Read: abstract.]

[10] V. W. Anelli, A. Bellogín, T. Di Noia, D. Jannach, C. Pomo. Top-N recommendation algorithms: a quest for the state-of-the-art. *UMAP*, 2022. arXiv:2203.01155. [Read: abstract.]

[11] M. Ludewig, D. Jannach. Evaluation of session-based recommendation algorithms. *User Modeling and User-Adapted Interaction*, 2018. arXiv:1803.09587. [Read: abstract.]

[12] M. Li, S. Jullien, M. Ariannezhad, M. de Rijke. A next basket recommendation reality check. arXiv:2109.14233, 2021. [Read: abstract and conclusions.]

[13] Y. Ji, A. Sun, J. Zhang, C. Li. A re-visit of the popularity baseline in recommender systems. *SIGIR*, 2020. arXiv:2005.13829. [Read: abstract.]

[14] Y. Ji, A. Sun, J. Zhang, C. Li. A critical study on data leakage in recommender system offline evaluation. arXiv:2010.11060, 2020. [Read: abstract.]

[15] Z. Meng, R. McCreadie, C. Macdonald, I. Ounis. Exploring data splitting strategies for the evaluation of recommendation models. *RecSys*, 2020. arXiv:2007.13237. [Read: abstract.]

[16] W. Krichene, S. Rendle. On sampled metrics for item recommendation. *KDD*, 2020. [Read as the preprint: S. Rendle, Evaluation metrics for item recommendation under sampling, arXiv:1912.02263, 2019; abstract.]

[17] A. Schein, A. Popescul, L. Ungar, D. Pennock. Methods and metrics for cold-start recommendations. *SIGIR*, 2002. [Read in full.]

[18] Choi et al. A recommender system for NFT collectibles with item feature. arXiv:2403.18305, 2024. [Read: abstract and conclusions.]

[19] S. Li, Y. Jin, P.-L. Hsu, Y.-S. Luo. NFT.mine: an xDeepFM-based recommender system for non-fungible token (NFT) buyers. arXiv:2306.03942, 2023. [Read: abstract and conclusions.]

[20] C. J. C. Burges. From RankNet to LambdaRank to LambdaMART: an overview. Microsoft Research Technical Report MSR-TR-2010-82, 2010. [Read: abstract and the sections on RankNet, LambdaRank and LambdaMART, without the derivations.]

[21] P. Covington, J. Adams, E. Sargin. Deep neural networks for YouTube recommendations. *RecSys*, 2016. [Read: abstract, system overview and conclusions.]